

Welcome back to the LangChain Masterclass! Pichli class mein humne seekha ki PDFs aur CSVs ko Document objects mein kaise load karte hain.

Lekin ab ek bohot badi practical problem aati hai: **Size**. Agar aap ek 500-page ki medical textbook LLM ko bhejenge, toh system crash ho jayega. Is problem ko solve karne ke liye hum **Text Splitters** (Chunking) ka use karte hain. Yeh Retrieval-Augmented Generation (RAG) ka sabse critical step hai.

Chaliye isko deeply samajhte hain!

Module 1: The Context Window Limitation (Topics 1, 2 & 3)

1. Why large documents cannot be sent directly & 2. Context Window

Har LLM (GPT-4, Gemini, Claude) ka ek "Context Window" hota hai. Context window ka matlab hai ki model ek baar mein kitne tokens (words/characters) padh aur yaad rakh sakta hai.

Agar ek model ki limit 8,000 tokens hai, aur aap usko 20,000 tokens ka PDF document bhejte hain, toh API strictly error throw kar degi: *"Token limit exceeded"*. Saath hi, agar aap bohot lamba text bhejte hain, toh LLM text ke beech ka hissa bhool jata hai (ise "Lost in the Middle" phenomenon kehte hain).

3. What is Chunking? (Real-world Analogy)

- **Analogy:** Imagine aapko ek poora bada Pizza (Large Document) khana hai. Aap poora pizza ek baar mein muh (Context Window) mein nahi daal sakte. Aapko usko Slices (Chunks) mein kaatna padega.
Chunking ka matlab hai ek lambe text ko chhote-chhote meaningful pieces mein todna, taaki baad mein hum sirf wahi pieces LLM ko bhejin jo user ke question se relevant hon.

Module 2: Text Splitters in Action (Topics 4, 6)

4. What are Text Splitters?

LangChain mein TextSplitters wo utility classes hain jo documents ko smart tarike se chunks mein todti hain. "Smart" isliye kyunki agar aap text ko blindly 100-100 characters mein kaatenge, toh ek word ya sentence beech mein se cut jayega (e.g., "LangC" ek chunk mein, "hain" doosre chunk mein), jisse meaning destroy ho jayega.

6. Chunk Size and Chunk Overlap

Text Splitters 2 main parameters par kaam karte hain:

1. **Chunk Size:** Ek chunk kitna bada hoga (e.g., 1000 characters).
2. **Chunk Overlap:** Do consecutive chunks ke beech mein kitna common (overlapping) text hoga.

Why Overlap? Agar ek sentence chunk ke border par cut ho gaya, toh overlap ensure karta hai ki agle chunk mein pichle chunk ki aakhiri line shamil ho, taaki context na toote.

- **Pizza Analogy for Overlap:** Jaise do pizza slices ke beech ka extra cheese thoda sa dono mein overlap karta hai, taaki taste continuous lage.

Flow Diagram:

Original Text: "LangChain is a framework. It is used to build AI apps. Text Splitters are great."

Chunk Size: 35 chars

Chunk Overlap: 10 chars

Chunk 1: "LangChain is a framework. It is use" (Wait, "use" is cut! Splitters fix this)

Smart Chunk 1: "LangChain is a framework. It is"

Smart Chunk 2: "work. It is used to build AI apps." (Notice the overlap of "work. It is")

Let's understand how chunk size and overlap work visually before writing the code.

Module 3: The Algorithms (Topics 5 & 7)

5. CharacterTextSplitter vs RecursiveCharacterTextSplitter

1. CharacterTextSplitter (The basic way):

Yeh strictly ek specific character (default `\n\n` - double newline) ke basis par split karta hai. Agar ek paragraph chunk size se bada ho gaya, toh yeh handle nahi kar pata.

2. RecursiveCharacterTextSplitter (The Industry Standard):

Yeh LangChain ka default aur sabse powerful splitter hai.

7. How RecursiveCharacterTextSplitter works internally

Yeh "Recursive" (step-by-step) hierarchy follow karta hai. Iski default separator list hoti hai: `["\n\n", "\n", " ", ""]`.

- **Step 1:** Yeh text ko paragraphs (`\n\n`) mein todne ki koshish karta hai.
- **Step 2:** Agar koi paragraph chunk_size se bada hai, toh yeh us specific paragraph ko sentences (`\n`) mein todta hai.
- **Step 3:** Agar koi sentence abhi bhi bada hai, toh usko words (space) mein todta hai.
- **Step 4:** Agar word bhi bada hai, tab jaakar characters (") par todta hai.
- **Result:** Isse paragraphs aur sentences ki natural boundary maintain rehti hai.

Module 4: Code Implementation

Let's build a Python application that uses RecursiveCharacterTextSplitter.

Complete Python Code & Dry Run

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

# 1. Dummy Large Text (Simulating a long document)
long_text = """
LangChain is a framework designed to simplify the creation of applications using LLMs.
It allows developers to connect language models to other sources of data.

Chunking is a very important part of this process. If you don't chunk, you will hit context limits.
Always ensure you have some overlap so sentences are not completely lost in translation.
"""

# 2. Initialize the Recursive Splitter
# In production, chunk_size is usually 500-1000. We use small numbers here for demonstration.
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,      # Maximum characters per chunk
    chunk_overlap=20,    # Number of characters overlapping between chunks
    separators=["\n\n", "\n", " ", ""] # The fallback order
)

# 3. Create a Document Object (Simulating output from a Document Loader)
doc = Document(page_content=long_text, metadata={"source": "langchain_guide.txt"})

# 4. Perform the Split
print("Splitting Document...\n")
chunks = text_splitter.split_documents([doc])

# 5. Output inspection
print(f"Total Chunks Created: {len(chunks)}\n")

for i, chunk in enumerate(chunks):
    print(f"--- Chunk {i+1} ---")
    print(f"Length: {len(chunk.page_content)} characters")
    print(f"Content: {chunk.page_content}")
```

```
print(f"Metadata: {chunk.metadata}\n")
```

Code Execution Dry Run:

Jab hum `split_documents` call karte hain, `LangChain` `long_text` ko dekhta hai. Yeh dekhta hai ki `chunk_size=100` hai. Yeh pehle `\n\n` (paragraphs) dhoondhta hai. Pehla paragraph 100 characters se bada hai, isliye wo agle separator `\n` aur fir spaces " " ka use karke words ke border par perfectly text ko split kar deta hai taaki koi word half-cut na ho. Har naya chunk original metadata (source: `langchain_guide.txt`) retain karta hai.

Module 5: Best Practices & Production Use (Topic 8)

Choosing the right chunk strategy & Best chunk sizes

- **General Text (Articles, Blogs):** `chunk_size = 1000`, `chunk_overlap = 200` characters.
- **Code Files (Python, Java):** `LangChain` provides a special `Language.PYTHON` splitter jo functions aur classes ke syntax ke basis par split karta hai (so a whole function stays in one chunk).
- **Production Consideration (TokenSplitters):** LLMs characters nahi, **Tokens** padhte hain (1 token $\approx$ 4 characters). Production mein hum `TokenTextSplitter` ya tiktoken based splitters use karte hain taaki hum exactly OpenAI ke API limits match kar sakein.

Review & Practice Segment

Interview Questions & Answers

Q: Why is `RecursiveCharacterTextSplitter` preferred over `CharacterTextSplitter`?

Answer: `CharacterTextSplitter` only splits at a single hardcoded character (like a newline). If a block of text lacks that character, it fails to split it, exceeding the chunk size.

`RecursiveCharacterTextSplitter` intelligently cascades through a list of separators (paragraphs -> sentences -> words -> characters), ensuring the chunk size is strictly respected while preserving the semantic structure (sentences/words) as much as possible.

Q: If you have a `chunk_overlap` of 50, but the total `chunk_size` is 100, what is the problem?

Answer: The overlap is too large relative to the chunk size (50%). The LLM will process highly redundant information, wasting token costs and making the context noisy. Overlap should ideally be around 10-20% of the chunk size.

Practice Problems

1. **Task 1:** Use the `CharacterTextSplitter` with `separator="."` to split a paragraph strictly by sentences. Set chunk size to 50. What happens if a sentence is 60 characters long? Test it.
2. **Task 2:** Write a script that loads your `server_logs.txt` (from the previous class) using `TextLoader` and then immediately passes those documents into a `RecursiveCharacterTextSplitter`. Print the length of the resulting chunk array.

Mini Assignment: The HealthBuddy Document Pre-processor

Aapke resume ke mutabiq aap "HealthBuddy" ecosystem par kaam kar rahe hain.

Ek pipeline banayein jo ek dummy Medical manual string ko load kare, aur usko 150 character chunks mein split kare with 30 character overlap, taaki next class mein jab hum in chunks ko

Vector Databases aur **Embeddings** mein push karein, toh wo perfectly ready hon!

Kya hum next topic mein Embeddings aur Vector Stores (jo AI ki long-term memory hote hain) shuru karein?